

The Coordinator Trap: Structural Scaling Liabilities in Mediated Multi-Agent Architectures and a Substrate-Based Alternative

Dr. Richard Nicholson

Tathata Systems Ltd, United Kingdom
tathatasystems@proton.me

June 2026

Preprint.

Abstract

Multi-agent coordination systems are converging on two dominant architectural patterns: *mediated hierarchies*, in which a central engine aggregates agent outputs, decides, and issues commands; and *registry-based discovery*, in which a global index brokers introductions between agents. Both patterns introduce a coordinator—by different names—that becomes a bottleneck, a single point of failure, and an unbounded audit obligation as agent populations grow. We argue that these failure modes are not implementation deficiencies but structural consequences of the coordinator assumption itself. Drawing on Holland’s framework for Complex Adaptive Systems, we propose an alternative in which coordination emerges from substrate properties rather than explicit protocols. We present Mycelium, a working implementation of this model as an embeddable Rust library, and show how its three-layer architecture—gossip KV store, signal mesh with boundary admission, and epidemic consensus—structurally addresses each failure mode rather than ameliorating it.

1. Introduction

A dominant family of AI agent frameworks released between 2023 and 2026—the supervisor, manager, hierarchical orchestration, and router patterns found in LangChain, AutoGen, CrewAI, and their successors—converges on the same structural pattern: a central orchestrator receives requests, routes them to specialist sub-agents, aggregates their outputs, and returns a synthesised result [1–3]. The orchestrator is the coordinator. The sub-agents are its workers.

This paper argues that the coordinator is not a solution to the multi-agent coordination problem. It is a restatement of the problem at a different scale.

The coordinator trap produces three failure modes that are structural, not incidental: (i) unbounded audit obligation, as every agent output passes through the coordinator and

generates an artifact of equivalent apparent authority regardless of relevance; (ii) context loss, as shared state held in or near the coordinator is lost on restart, forcing agents to re-litigate settled decisions; and (iii) output format mismatch, as the coordinator broadcasts undifferentiated results to heterogeneous consumers it has no model of.

These failure modes have been documented in production deployments. In the Cisco Mycelium project—an unrelated multi-agent orchestration system—the coordinator could not maintain context across restarts, produced three-hour and five-second problems with the same apparent authority, and generated structured machine logs that failed to communicate to human reviewers [4, 5].

We show that these failure modes are irreducible by improving the coordinator. They follow necessarily from its existence. We then derive a coordinator-free architecture from John Holland’s signal/boundary model [6] and present Mycelium, a working implementation as an embeddable Rust library.

2. Background: The Two Dominant Patterns

2.1 Mediated Hierarchy

A mediated hierarchy places a cognitive engine at the centre of the agent population. Specialist agents receive structured payloads from the engine, process them, and return results. The engine aggregates, decides, and broadcasts. Agent populations in contemporary LLM frameworks [2] follow this pattern: the orchestrator holds the conversation state and the routing logic; sub-agents are workers in a fanout RPC system.

Kubernetes [7] represents a more mature instance of the same architectural pattern. A control plane holds the declared target state; reconciliation loops continuously compare actual cluster state to that target and issue corrective actions. The coordinator is more sophisticated than a negotiation engine, but the structural relationship is identical: state is held above the runtime, and correctness is enforced top-down.

2.2 Registry-Based Discovery

A registry-based system separates coordination from discovery. Agents register capabilities in a global index; other agents query the index to locate peers. The registry brokers introductions; agents communicate point-to-point thereafter. FIPA’s Directory Facilitator [8, 9] is a canonical instance. MIT NANDA [10] represents a more sophisticated point on the same spectrum: it uses CRDT-based updates, cryptographically verifiable AgentFacts, and multi-endpoint routing rather than a static index. This spectrum runs from static centralised registries through federated and CRDT-verifiable designs to fully local self-admission substrates. The claim here applies to the common structural property they share: an authority-bearing component through which discovery passes and which holds canonical state about the agent population. Mycelium sits at the far end of this spectrum—no registry, no authority, no discovery component—but the critique does not apply equally to every design in the family.

2.3 The Shared Assumption

Superficially, these two patterns are different. A mediated hierarchy coordinates behaviour at runtime; a registry brokers introductions at discovery time. But they share a single structural assumption: **there exists a designated component that holds authoritative state about the agent population and through which coordination passes.**

This assumption is the coordinator trap. Section 4 traces each failure mode to this assumption directly.

2.4 The Separation Tax

The coordinator assumption has a concrete operational cost. A representative conventional ML inference pipeline must address at minimum four distinct infrastructure concerns: task dispatch and queuing (Redis/Celery or Kafka depending on scale), worker registration and discovery (Consul, etcd, or a sidecar), failure recovery and dead-letter handling, and cross-component observability. Each concern requires a separately deployed, operated, and monitored service. The pipeline is not the model—it is the scaffolding around the model.

Each concern is a separate operational surface: a separate failure mode, a separate scaling policy, a separate upgrade cycle, and a conceptual translation layer. The coordination work—route an item to a capable worker, throttle when workers are saturated, clean up abandoned items, make topology visible—is the same abstract work at every level. The *separation tax* is the cost of performing that work redundantly at each concern boundary rather than once in a unified substrate.

The two dominant patterns described in §§2.1–2.2 do not reduce this tax. They add another coordination system on top of the existing stack. This paper’s claim is that the separation tax is not a necessary cost of multi-agent coordination. It is the cost of applying the coordinator assumption independently at each layer of the stack.

3. A History of the Coordinator Assumption

The coordinator assumption has been the default in distributed computing for over fifty years. Each generation has recognised the problems it causes and attempted to mitigate them without eliminating the assumption itself.

3.1 Blackboard Systems (1970s–80s)

The blackboard model, most prominently realised in HEARSAY-II [11], organises coordination around a central shared data structure and a set of specialised knowledge sources that read from and write to it. A control component monitors the blackboard and decides which knowledge source to activate next. HEARSAY demonstrated that the model could solve hard problems; it also demonstrated the failure modes: the control component became a bottleneck and a single point of failure, and the blackboard accumulated stale entries.

3.2 The Actor Model (Hewitt, 1973)

Hewitt, Bishop and Steiger’s Actor model [12] provided the first formal model of concurrent computation as communicating entities. The model eliminates shared mutable state—a genuine advance—but retains explicit addressing: to send a message, an actor must know

the recipient’s address. Topology must be managed explicitly by the sender. The Actor model correctly identified agents as the unit of composition; it did not identify boundaries as the mechanism for self-organising communication.

Mature actor runtimes—most notably Erlang/OTP—address fault tolerance and process supervision without a central coordinator, and represent a genuine step beyond the original formulation. The addressing problem persists, however: Erlang’s process registry is a coordinator with a narrower mandate, and OTP supervision trees impose a coordinator hierarchy for failure propagation.

3.3 Linda and the Tuple Space (Gelernter, 1985)

Gelernter’s Linda [13] introduced generative communication via a shared tuple space. Carriero and Gelernter [14] established that coordination is a distinct language design concern, separable from computation—a correct and underappreciated insight. What Linda retained, however, is a shared coordinator: the tuple space itself must remain accessible, consistent, and available to all participants. Fault-tolerant variants required replication schemes and consistency protocols. The medium is the coordinator. The coordinator trap persists in a subtler form.

Carvalho [15] observes that modern AI agent frameworks are independently rediscovering Linda’s tuple space pattern—implementing ad-hoc shared stores with task claiming, polling, and matching—without awareness of forty years of research on its failure modes. The community is not learning from history. It is repeating it.

3.4 BDI Agents and FIPA Standards (1990s)

The Belief-Desire-Intention agent architecture [16] and the Foundation for Intelligent Physical Agents [8, 9] brought deliberative reasoning to multi-agent systems. FIPA defined a Directory Facilitator—a registry where agents register capabilities and query for peers—that is precisely the registry-based pattern that MIT NANDA echoes today, four decades later.

3.5 The Pattern That Persists

Across five decades, each model correctly identified coordination as a concern distinct from computation. None eliminated the coordinator assumption:

Model	Coordinator retained
Blackboard / HEARSAY (1980)	Central blackboard + control component
Actor model (Hewitt, 1973)	Explicit addressing—sender must know recipient
Linda (Gelernter, 1985)	Shared tuple space
BDI / FIPA / JADE (1990s)	Directory Facilitator as registry
Modern LLM orchestration (2020s)	Central orchestrator / CognitiveEngine

4. The Coordinator Trap: From Architecture to Symptom

We use the term *coordinator trap* to describe the set of failure modes that any architecture inheriting the coordinator assumption must exhibit. These are not bugs; they are structural properties. A better coordinator does not escape them.

4.1 The Mediated Hierarchy Described Precisely

In a mediated hierarchy, agents are workers in a fanout RPC system. The coordinator aggregates responses, applies its decision logic, and broadcasts results. When we call these components “agents” we are importing a concept from the Complex Adaptive Systems literature. Holland [6] defines an agent as an entity that holds a *boundary*—a receptor set specifying the conditions under which it acts on incoming signals. The boundary is intrinsic to the agent, not delegated from outside. However, in a mediated hierarchy, agents satisfy none of this definition: they receive what the coordinator routes, not what they declare themselves competent to receive. They are passive processors of coordinator-routed payloads.

4.2 Audit Burden: The Consequence of Post-Admission Broadcasting

The unbounded audit obligation follows directly from the coordinator’s broadcast model. Every output of every agent passes through the coordinator and produces a coordinator-level artifact with identical structure and apparent authority, regardless of the quality or validity of the underlying agent reasoning. This is post-admission filtering: artifacts are produced first; quality is assessed after. As agent populations grow, the coordinator’s cognitive load grows linearly with the number of domains it must evaluate. The audit burden scales with the coordinator. The three-hour problem and the five-second problem have the same voice because the coordinator processes them identically [4].

4.3 Context Loss: The Consequence of Coordinator-Held State

In a mediated hierarchy, shared state lives in or near the coordinator. Agents are stateless workers; they hold no durable view of prior decisions. When the coordinator restarts, the state is gone. When a new agent joins, it must query the coordinator for a catchup briefing. The attempted fix documented in [5]—a filesystem-based shared memory—correctly identifies that state should be held outside the coordinator, but introduces a shared mutable store with no eviction policy: entries persist indefinitely, accumulate, and drift from current system state. This is not a solution to the coordinator trap; it is a redistribution of the same problem.

4.4 Output Format Mismatch: The Consequence of Absent Boundary Model

When the coordinator synthesises a result and broadcasts it, it has no model of its recipients’ boundaries—no representation of which consumers can act on which signals. All downstream consumers receive the same output because, from the coordinator’s perspective, they are all equivalent consumers of its decisions. A human reviewer is not equivalent to an agent as a consumer of a coordination decision, but the coordinator cannot know this without a model of recipient boundaries.

4.5 The Category Error: Workers Dressed as Agents

The three failure modes share a common root: in a mediated hierarchy, components called “agents” are not agents in any theoretically meaningful sense. A component whose domain is determined by the coordinator’s routing decisions is a worker node in a fanout RPC system. Calling these components “agents” inherits none of the architectural properties that make

genuine agents scalable.

The exit from the coordinator trap requires not a better coordinator but the elimination of the coordinator from the substrate.

5. Theoretical Foundation: Holland’s Signal/Boundary Model

John Holland’s framework for Complex Adaptive Systems [6, 17], formalised in *Signals and Boundaries: Building Blocks for Complex Adaptive Systems* (MIT Press, 2012), provides the theoretical basis for a coordinator-free architecture.

Holland’s thesis: the behaviour of complex adaptive systems emerges from two and only two primitives.

Signals propagate through a medium unconditionally. No signal is withheld from propagation on the basis of who might act on it.

Boundaries are receptor sets. Each agent holds a boundary—a set of conditions under which it *acts* on a signal. Forwarding is always unconditional; acting is always conditional on boundary admission.

This inversion is the key insight. Conventional distributed systems thinking routes messages to known recipients; the emitter must know who is listening. In Holland’s model, any agent whose boundary matches a signal responds. The emitter does not need to know who is listening. The system tolerates churn without stalling because there is no routing table to maintain, no coordinator to query.

Stigmergy and TTL evaporation. Holland identifies stigmergy—state left in the medium by agents, readable by other agents—as the mechanism through which agents coordinate without direct communication [18]. Applied to distributed state management, this suggests that shared state should carry a time-to-live and evaporate unless actively refreshed. An agent that is alive and relevant keeps its state fresh; a dead agent’s state simply fades. Failure is emergent: the absence of refreshment *is* the failure signal.

Applied to the failure modes of Section 4: pre-admission filtering structurally prevents the audit burden—an agent whose boundary does not match a signal produces no artifact to audit. Distributed TTL state makes coordinator-type context loss structurally highly improbable—there is no coordinator memory to lose; state evaporation under simultaneous node failure remains possible but requires active failure of TTL renewal across all replicas simultaneously. Self-differentiating boundaries make output format mismatch a design problem rather than a routing obligation. Each failure mode is not ameliorated; it is either structurally prevented or made structurally highly improbable at the substrate level.

6. Prior Art: Correct Concepts, Wrong Implementation

Four prior systems identified the correct underlying concepts but implemented them with too much protocol ceremony, or in the wrong deployment model, to achieve their full potential.

6.1 Jini and the Lease Insight (Sun Microsystems, 1998)

Sun Microsystems’ Java/Jini [19, 20] was designed to address runtime dynamism: in a live network, services appear, depart, and change in ways no deployment manifest can anticipate, and a crashed service cannot explicitly deregister. The lease was Jini’s answer—decay is the right primitive for an unpredictably dynamic world. A service holds a lease on its registration; if it does not renew, the registration expires, providing implicit failure detection without requiring an explicit deregistration protocol. The insight is correct. The implementation was protocol-heavy: explicit `Lease` objects, `renew()` RPCs, a lease manager, explicit cancellation. The ceremony obscures the substrate property—that registrations should evaporate unless actively maintained—behind an explicit lifecycle protocol.

6.2 OSGi Requirements and Capabilities

OSGi’s insight was that software agility is a function of modular, dynamically assembleable components. The OSGi Alliance [21] formalised this as a dependency model in which software modules declare capabilities they provide and requirements they need; a resolver matches providers to consumers. The primitive is correct: declarative matching between providers and consumers, with the resolver handling wiring.

What mainstream OSGi adoption got wrong was treating resolution as static—performed once at bundle-install time. A module that disappeared at runtime left a gap with no in-process mechanism for repair; closing it required a human to rebuild and redeploy. Dynamic assembly was the goal; deploy-time resolution was where that ambition stopped. Paremus Service Fabric addressed this directly.

6.3 Paremus Service Fabric and the Reconciliation Engine

Paremus Service Fabric (circa 2010–2015) [22] began as a Jini runtime with OSGi components, using Jini’s service discovery as the underlying discovery mechanism. Over time, Jini service discovery was replaced with the OSGi Remote Services specification backed by an in-house gossip-based implementation—decentralised discovery, TTL-managed registrations, no central lookup service. What emerged from this evolution was a deliberate fusion of Jini’s runtime dynamism with OSGi’s modular assembly model: the R&C graph as a *declared target topology*, with resolution made continuous against a live, gossip-discovered environment rather than a static deployment manifest. The runtime was continuously monitored against that declared target state and deltas driven back into convergence—closer to the Kubernetes control loop than to a shared knowledge graph, and requiring no human redeploy.

What did not change was the hermetic relationship between the gossip layer and the reconciliation engine: ‘as above, so below’—the declared target state lived in the central engine above; the gossip layer revealed the landscape below; the engine held the authority to reconcile the two. The coordinator trap persisted not in the discovery mechanism but in the architectural layer that held the target.

What Paremus still required was a *central reconciliation engine* holding that target state. If it went down, reconciliation stopped. The coordinator trap in a more sophisticated form.

6.4 Flow-Based Programming (Morrison, c. 1971)

Flow-Based Programming [23], developed by J. Paul Morrison at IBM in the early 1970s, conceived programs as networks of independent black-box processes communicating through bounded connections. The model separates coordination from computation and gets three things right: (i) *reusable processes* that know nothing about their neighbours; (ii) *automatic backpressure*, in which bounded connections block writers when full; and (iii) *declarative topology* expressed as a data structure separate from component code.

What FBP retained was an external medium to which all these properties belong. The FBP runtime *is* the coordinator: it allocates connections, manages backpressure, and resolves the wiring. The coordinator trap remains.

6.5 The Strip-the-Ceremony Pattern

A pattern emerges across all four cases:

Prior art	Correct concept	Implementation ceremony	Substrate property equivalent
Jini	Registrations should decay	<code>Lease.renew()</code> RPC + lease manager	TTL as natural evaporation
OSGi	Declarative capability matching	Static bundle-install resolver	Continuous evaluation against live state
Paremus	Continuous reconciliation toward target state	Central reconciliation engine + target state graph	Gossip mesh — every TTL refresh is a reconciliation tick
FBP	Backpressure, declarative topology, reusable processes	External FBP runtime managing connections	Opacity KV flag — agent self-declares saturation; resolvers skip it

The pattern that produces better architecture: identify the correct concept, find the substrate *property* that produces the same behaviour without an explicit protocol, implement the property and let the behaviour emerge.

7. Mycelium: A Coordinator-Free Substrate

Mycelium is an embeddable Rust library implementing Holland’s signal/boundary model. It has no daemon, no control plane, no installer, no orchestrator. It is a crate embedded in the caller’s process. The architecture is three layers; each directly addresses one of the three structural failure modes identified in Section 4: audit burden (§4.2), context loss (§4.3), and output format mismatch (§4.4).

7.1 Layer I — KV Store: Addressing Context Loss

Layer I is a gossip-replicated key-value store. Every entry carries a Hybrid Logical Clock timestamp for causal last-write-wins ordering and a time-to-live. Entries are replicated across the cluster via epidemic gossip; anti-entropy reconciliation runs on peer reconnection. There is no primary, no leader, no coordinator. Every node holds a full replica of the key

namespace relevant to it.

Context loss structurally highly improbable. Context loss in the mediated hierarchy arises because shared state is held in or near the coordinator. In Mycelium’s Layer I, there is no coordinator memory to lose. A node that restarts reconnects to its peers, runs anti-entropy, and recovers full mesh state within one gossip cycle. State loss requires simultaneous TTL expiry across all replicas—possible under full-cluster failure, but structurally different from the single-point loss inherent in coordinator designs.

TTL evaporation as stigmergy. Every capability advertisement, every group membership record, every opacity flag is a KV entry with a TTL. A live node continuously re-advertises; the trail stays fresh. A dead node stops re-advertising; its entries evaporate. There is no explicit failure detection protocol because absence of refreshment *is* the failure signal.

The key inversion from Paremus. In Paremus, the target state was held *above* the runtime in a central graph and pushed down into components. In Mycelium, the target state is *compiled into the runtime components themselves*—each node carries its own fragment as capability and requirement declarations. There is nothing external to converge toward. Convergence emerges upward from those fragments.

7.2 Layer II — Signal Mesh: Addressing Audit Burden and Output Mismatch

Layer II is an ephemeral signal mesh implementing Holland’s signal/boundary model directly. Signals are emitted with a scope and propagated unconditionally to all reachable nodes. At each receiving node, `Boundary::admits()` evaluates the signal against the node’s declared receptor set. Only nodes whose boundary matches the signal act on it. Forwarding is always unconditional; acting is always conditional on boundary admission.

Audit burden structurally prevented. A signal scoped to the `gpu-compute` group only activates nodes in that group. Signals propagate at the transport layer unconditionally—that is the substrate’s routing mechanism—but an agent in an unrelated group produces no *application-layer artifact*: it does not act, does not log, does not emit a downstream result. The audit burden that scales with a coordinator scales because every event passes through a single processing point that generates a record regardless of relevance. Here, the record is never created because the admission test fails before processing begins. The three-hour problem and the five-second problem never land on a reviewer who cannot evaluate them.

Output format mismatch structurally addressed. A signal emitted to a `human-review` group reaches only nodes whose boundary includes human-review membership; a signal emitted to `automated-pipeline` reaches a different set. Human-facing and agent-facing consumers self-differentiate via their boundary declarations without any routing logic in the emitter. The representation design obligation—what a human-review signal should *contain*—remains with the emitting agent; the substrate ensures it reaches the right consumers, not that its content is appropriate for them.

7.3 Capability and Requirement Declarations: Addressing the Category Error

Each Mycelium node holds a `CapabilityGroupDef`—a declarative specification of capabilities it provides and requirements it needs. The node independently evaluates whether it should join each capability group by testing its own capabilities against the group’s filter, right now,

against live KV state. Membership is self-assigned, not coordinator-assigned. The node *is* its boundary. The application’s coordination topology emerges from the composition of these declarations across the live node population, not from a coordinator’s routing table.

7.4 Layer III — Consensus: Addressing the Coordinator Bottleneck

Layer III provides epidemic consensus—`group_propose`, `system_propose`, and `cross_group_propose`—implemented entirely through the signal mesh. `PROPOSE`, `VOTE`, and `COMMIT` are signal payloads riding ordinary Layer II frames. The substrate is unaware that consensus is happening.

Fault model and guarantees. The consensus layer is crash-fault tolerant; it does not provide Byzantine fault tolerance. A ballot commits when a configurable quorum fraction of group members—defaulting to a simple majority—has voted. A node that crashes mid-ballot simply stops voting; the ballot either reaches quorum without it or times out. Under network partition, liveness is not guaranteed: a minority partition will not commit. Safety—no two nodes commit conflicting values for the same ballot—holds as long as quorum is not simultaneously satisfied by two disjoint subsets, which a majority quorum prevents. Quorum membership is evaluated against the live capability-group population at ballot time, not a fixed initial roster; nodes that join or leave between proposal and commit may affect whether quorum is reachable, but cannot cause two conflicting values to commit. This is not a replacement for Raft, Paxos, or BFT consensus in high-assurance settings; it is a coordination primitive for capability-group decisions in homogeneous trusted clusters where crash-fault tolerance and leaderless operation are the primary requirements.

`cross_group_propose` supports ballots where multiple named capability groups act as independent voting blocs. A proposal commits only when every group independently reaches its required quorum fraction. This supports multi-AZ durability requirements, compliance ratification, and hierarchical AI pipelines—all without a coordinator.

Coordinator bottleneck structurally addressed. In a mediated hierarchy, coordination throughput is bounded by the coordinator’s capacity, and coordinator failure stops coordination entirely. In Layer III, there is no coordinator. No single node’s failure prevents other ballots from proceeding. The system degrades gracefully to reduced quorum, not coordination failure.

7.5 Agentic Flow Networks

The three-layer architecture described in §§7.1–7.4 supports a distinct multi-agent computational pattern that we term an *Agentic Flow Network* (AFN). An AFN is a topology of capability-bounded agent groups in which work flows through the mesh as a sequence of KV state transitions, each stage consuming from one key prefix and producing to the next.

An AFN has five structural properties:

1. **Substrate Unity.** All coordination—capability advertisement, group membership, backpressure, work items, completion signals—is carried by the same gossip KV substrate. There is no separate message queue, registry, or coordination bus.
2. **Topology Emergence.** Which nodes participate in which pipeline stages is not

configured by an operator. Each node evaluates its own capabilities against the group definitions and self-joins the matching stages.

3. **Fluid Allocation.** There is no static assignment of workers to stages. Any node capable of serving a stage joins its group; the group’s size fluctuates with available capacity.
4. **Opacity Backpressure.** A saturated node writes an opacity flag to `sys/load/{self}/capacity` (TTL ≈ 5 s), making itself invisible to resolvers at the next stage. The topology self-throttles without any flow-control protocol.
5. **TTL-Native Cleanup.** Work items written to the KV substrate carry TTLs. Abandoned items evaporate automatically. There is no dead-letter queue, no explicit cancellation protocol.

These five properties are validated empirically by `examples/fluid_pipeline/` —a 10-worker, 4-stage news article pipeline (fetch $\rightarrow$ parse $\rightarrow$ score $\rightarrow$ publish) running over a shared KV ring as distributed buffer, exercised end-to-end by integration scenario 11 (repository and test harness: §8).

An AFN is not a new programming model layered on top of Mycelium. It is the name for the pattern that naturally arises when a distributed application is built correctly on a substrate that eliminates the coordinator.

8. Evaluation

Mycelium’s correctness across its three-layer architecture is validated by 243 unit tests and 12 integration scenarios run against a live 5-node Docker cluster. Scenarios cover KV replication under partition and reconnection, signal delivery and boundary admission, capability group formation and dissolution, consensus quorum under node failure, cross-group voting, the full Agentic Flow Networks pipeline, and Prompt Skills cross-node KV propagation with LLM invocation. All 12 scenarios pass at HEAD.

The implementation is publicly available at <https://github.com/RichardEko/mycelium> (commit 1717d4e, tag `paper-submission-v1`) under the AGPL-3.0 licence. The integration test harness is included in the repository and reproducible with a single `make test` invocation against a Docker-composed 5-node cluster. The AGPL-3.0 licence is intentional: applications that deploy Mycelium as part of a networked service must publish their source under the same terms, closing the SaaS loophole that permissive licences leave open. Organisations requiring proprietary embedding may obtain a commercial licence from Tathata Systems Ltd.

9. Discussion

9.1 The Hayek Parallel

The coordinator trap is not a new discovery. Friedrich Hayek described it in 1945—for economies.

In *The Use of Knowledge in Society* [24], Hayek argued that the central planning problem is

not computational—it is epistemic. No central planner can possess the distributed, local, tacit knowledge held by individual market participants. Prices are not just numbers; they are signals that aggregate and propagate dispersed knowledge through the economy without anyone needing to understand the whole.

The parallel to the coordinator trap is suggestive rather than exact—economies, biological systems, and distributed software are not identical. But all three expose the same cost of centralising knowledge that is generated and updated at the edge. A planned economy and a mediated hierarchy share a recognisable failure mode: a central node that must aggregate knowledge it cannot fully possess, synthesise decisions on behalf of participants who understand their local context better than any coordinator can, and issue commands downward into a system whose ground truth is always more current at the edges than at the centre.

Hayek’s market is a signal/boundary system. He just did not have Holland’s vocabulary.

9.2 The Beinhocker Parallel: Organisations as Complex Adaptive Systems

Beinhocker’s *The Origin of Wealth* [25] reaches the same conclusion from a third direction: organisations. Drawing on extensive empirical analysis of corporate performance, Beinhocker finds that hierarchical organisations centralising strategic information and decision-making systematically underperform distributed ones—not because their leaders are less capable, but because the local knowledge required for adaptive decisions cannot travel up the chain fast enough to remain actionable by the time it arrives.

Hayek, Holland, and Beinhocker are describing the same property of complex adaptive systems at three different scales—economies, biological systems, and organisations. The coordinator trap is not a software engineering peculiarity. It is a structural consequence of centralising coordination in any sufficiently complex adaptive system.

9.3 The Economic Case: Quadratic Cost Decomposition

The architectural argument above is independent of cost. There is also a direct economic argument. For workloads in which context accumulates across iterations—the common case in long-running coordinator pipelines where each step appends to a growing history—LLM agent execution cost scales superlinearly with iteration count. Because each API call re-processes the entire accumulated context, the token count at iteration k is $\Delta T + \tau k$ (where ΔT is the initial prompt and τ is context growth per iteration). Summing over k iterations yields a total proportional to k^2 : cost is quadratic in iteration count, not linear. Systems that summarise, truncate, or selectively retrieve context can reduce but not eliminate this accumulation; the model below applies at its strongest to pipelines that maintain a running context window.

The decomposition leverage is direct. Splitting a k -iteration task into M independent subtasks of k/M iterations each reduces total cost by factor M :

$$M \times \left(\frac{k}{M} \right)^2 = \frac{k^2}{M}$$

versus k^2 for the monolithic run. A 4-way decomposition of a 100-iteration task costs

$4 \times 25^2 = 2,500$ versus $100^2 = 10,000$ —a $4\times$ saving from a $4\times$ decomposition.

A mediated hierarchy cannot realise this saving without explicit orchestration overhead. The coordinator’s synthesis step, where it aggregates M subtask results into a combined output, recreates the full accumulated context and pays the quadratic penalty once more. However, Mycelium’s capability-bounded groups implement decomposition automatically: each group’s TTL constrains how long a work item remains live in that stage, which under typical workloads approximates a bound on effective k for that group—a time bound rather than a strict iteration count bound, but one that produces the same practical effect when per-iteration latency is roughly constant. The quadratic cost accumulation resets at each group boundary, and no coordinator synthesis is required.

The practical significance of this saving is reinforced by empirical energy measurements. White [26] demonstrates that inference energy per query varies by two orders of magnitude across model families and sizes, and that energy cost is highly sensitive to input characteristics. Where the quadratic accumulation compounds an already expensive per-call baseline, the saving from decomposition is not a theoretical nicety—it is an operational necessity.

9.4 Limitations

Mycelium assumes a cluster the operator owns. Mycelium’s scope is intra-cluster; cross-organisational discovery requires external registry infrastructure. Mycelium’s conforming A2A endpoint makes it reachable from any A2A-speaking registry without modification to either side.

Ephemeral signals are intentionally not durable—a node that misses a signal misses it. Durable delivery is a higher-order concern built on the KV layer or consensus, not a substrate property.

Boundary admission requires agents to declare their boundaries correctly. A misconfigured boundary—too broad or too narrow—produces incorrect routing without any coordinator to catch the error. This places a correctness obligation on the capability declarations that the mediated hierarchy places on the coordinator instead.

Trust and adversarial behaviour. Self-declared capability boundaries raise questions about false capability claims, malicious group admission, and poisoned KV entries. Mycelium’s intra-cluster assumption—the operator owns the cluster—substantially reduces this surface, but owned clusters still contain buggy or compromised nodes. The `tls` feature addresses node identity: each node holds an Ed25519 keypair; mTLS authenticates every TCP connection; KV writes under the `tls` feature are Ed25519-signed at the frame level (wire version 10), making undetected mutation of replicated state infeasible for an attacker who has not compromised the originating node’s private key. The current implementation does not provide capability-claim attestation—a node can declare capabilities it does not possess—and Byzantine-tolerant consensus is out of scope. These are correct tradeoffs for the target deployment model (trusted, operator-controlled clusters) but should be revisited before Mycelium is used in adversarial multi-tenant environments.

The quadratic cost decomposition in §9.3 assumes that the k -iteration task can be partitioned into M independent subtasks. Many real agentic workloads are inherently sequential—multi-

step reasoning chains, document drafting, iterative refinement—where each step depends causally on the prior and decomposition is not valid. The saving applies where subtask independence holds; it does not apply universally.

When not to use AFN. Five cases warrant a different approach: *Strict ordering*—where tasks must be processed in a guaranteed sequence, not by the first available worker. *Exactly-once delivery*—where each work item must be processed exactly once, with no duplicate execution under any failure scenario. *Complex DAG dependencies*—where later stages must block on combinations of earlier results in ways that cannot be expressed as linear KV-prefix pipelines. *Long-term log retention*—where TTL evaporation is a correctness property; applications requiring replay capability must use a separate append-only store. *Cross-cluster fan-out at scale*—where the gossip mesh operates within a single cluster; multi-cluster deployments require a federation layer.

9.5 Future Work

- **Empirical comparison** against a deployed mediated hierarchy at equivalent agent counts. Key measurements: (i) coordination convergence time—single-ballot `group_propose` vs NegMAS [27] SAO N-round negotiation; (ii) failure tolerance—coordinator failure in a mediated hierarchy vs random node failure in Mycelium; (iii) state freshness under churn—TTL evaporation latency vs knowledge-graph drift rate; (iv) audit obligation under load—artifact count growth as agent population scales.
- **Formal verification** of the signal/boundary substrate properties using TLA+ or similar.
- **Cross-cluster federation**—practical experience deploying multiple independent Mycelium clusters registered with an internet-scale A2A gateway [10], measuring discovery latency and trust propagation across organisational boundaries.
- **Signal reorder buffer**—receiver-side per-(sender, kind) HLC-keyed causal delivery for applications requiring strict signal ordering.

10. Conclusion

The coordinator is not a solution to the multi-agent coordination problem. It is a restatement of the problem at a different scale. Mediated hierarchies make the coordinator responsible for filtering, memory, routing differentiation, and fault tolerance—properties that in a correctly designed substrate belong to the agents themselves.

Unbounded audit obligation, context loss across restarts, and output format mismatch between heterogeneous consumers are not bugs in specific implementations. They are structural consequences of the coordinator assumption, predictable from first principles, and irreducible by improving the coordinator.

Holland’s signal/boundary model provides the theoretical basis for a different architecture: one in which coordination emerges from substrate properties rather than explicit protocols. Mycelium implements this model as an embeddable library. Each of its three layers—gossip KV store, signal mesh with boundary admission, epidemic consensus—addresses a mirrored

failure mode, not by handling the failure gracefully but by making each failure mode structurally prevented or, where substrate constraints apply, structurally highly improbable.

The target state for a Mycelium application is not a document held in a registry or a graph maintained by a reconciliation engine. It is the aggregate of what every node declares itself to be, compiled into the runtime components at build time, assembled bottom-up by the mesh at runtime, and always current because anything not actively maintained evaporates. Coordination emerges. No coordinator required.

The coordinator trap is not a new discovery. Hayek described the epistemic cost of centralisation for economies in 1945. Holland formalised the signal/boundary model for complex systems in 2012. Mycelium demonstrates that both insights transfer to AI agent fleets. The claim is the same in all three cases: distributed local knowledge, expressed through signals and boundaries, produces emergent order that no single coordination point can replicate—because the knowledge required is always more current at the edges than at the centre.

References

- [1] Yuzhe Zhang, Feiran Liu, Yi Shan, Xinyi Huang, Xin Yang, Yueqi Zhu, Xuxin Cheng, Cao Liu, Ke Zeng, Terry Jingchen Zhang, and Wenyuan Jiang. Silo-Bench: A scalable environment for evaluating distributed coordination in multi-agent LLM systems. *arXiv preprint arXiv:2603.01045*, 2026.
- [2] Lijun Sun, Yijun Yang, Qiqi Duan, Yuhui Shi, Chao Lyu, Yu-Cheng Chang, Chin-Teng Lin, and Yang Shen. Multi-agent coordination across diverse applications: A survey. *arXiv preprint arXiv:2502.14743*, 2025.
- [3] Rui Li, Zeyu Zhang, Xiaohe Bo, Quanyu Dai, Chaozhuo Li, Feng Wen, and Xu Chen. Towards adaptive, scalable, and robust coordination of LLM agents: A dynamic ad-hoc networking perspective. *arXiv preprint arXiv:2602.08009*, 2026.
- [4] Julia Valenti. Signal to noise. <https://juliavalenti.com>, April 2026.
- [5] Julia Valenti. The context problem nobody talks about with multi-agent teams. <https://juliavalenti.com>, 2026.
- [6] John H. Holland. *Signals and Boundaries: Building Blocks for Complex Adaptive Systems*. MIT Press, 2012.
- [7] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. *ACM Queue*, 14(1), 2016.
- [8] FIPA. FIPA agent management specification. Technical report, Foundation for Intelligent Physical Agents, 2004.
- [9] Fabio Bellifemine, Giovanni Caire, and Dominic Greenwood. *Developing Multi-Agent Systems with JADE*. Wiley, 2007.

- [10] Ramesh Raskar et al. Unlocking the internet of AI agents via the NANDA index. *arXiv preprint arXiv:2507.14263*, 2025.
- [11] Lee D. Erman, Frederick Hayes-Roth, Victor R. Lesser, and D. Raj Reddy. The HEARSAY-II speech-understanding system: Integrating knowledge to resolve uncertainty. *ACM Computing Surveys*, 12(2):213–253, 1980.
- [12] Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular ACTOR formalism for artificial intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 235–245, 1973.
- [13] David Gelernter. Generative communication in Linda. *ACM Transactions on Programming Languages and Systems*, 7(1):80–112, 1985.
- [14] Nicholas Carriero and David Gelernter. Coordination languages and their significance. *Communications of the ACM*, 35(2):97–107, 1992.
- [15] Otavio Carvalho. Our AI orchestration frameworks are reinventing linda (1985). <https://otavio.cat>, 2025.
- [16] Anand S. Rao and Michael P. Georgeff. BDI agents: From theory to practice. In *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS)*, pages 312–319, 1995.
- [17] John H. Holland. *Complexity: A Very Short Introduction*. Oxford University Press, 2014.
- [18] Marco Dorigo and Thomas Stützle. *Ant Colony Optimization*. MIT Press, 2004.
- [19] Jim Waldo. Jini architectural overview. Technical report, Sun Microsystems, January 1999.
- [20] Ken Arnold, Bryan O’Sullivan, Robert Scheifler, Jim Waldo, and Ann Wollrath. *The Jini Specification*. Addison-Wesley, 1999.
- [21] OSGi Alliance. OSGi core release 8 specification, chapter 27: Capabilities and requirements. Technical report, OSGi Alliance, 2020.
- [22] Paremus Ltd. Paremus service fabric. Continuous OSGi Requirements & Capabilities resolution; direct conceptual predecessor to Mycelium’s capability resolver, 2015. 2010–2015.
- [23] J. Paul Morrison. *Flow-Based Programming: A New Approach to Application Development*. CreateSpace, 2nd edition, 2010. First developed as an internal IBM paper, c. 1971.
- [24] Friedrich A. Hayek. The use of knowledge in society. *American Economic Review*, 35(4):519–530, September 1945.
- [25] Eric D. Beinhocker. *The Origin of Wealth: Evolution, Complexity and the Radical Remaking of Economics*. Harvard Business School Press, 2006.

- [26] Melvyn J. White. Inference scaling laws for mathematical code generation: Family-specific behavior and energy analysis. https://quantumzxxxyy.github.io/quantumzxxxyy/inference_scaling_final.pdf, November 2025.
- [27] Yasser Mohammad, David Viqueira, Alvaro Ayerbe, and Ido Kissos. NegMAS: A platform for automated negotiations. <https://negmas.readthedocs.io>, 2020.
- [28] Richard Nicholson. Modularity. OSGi Alliance / Eclipse Foundation. <https://osgi.org/resources/modularity/>, .
- [29] Richard Nicholson. Complexity, modularity and business. OSGi Alliance / Eclipse Foundation. <https://osgi.org/resources/complexity-modularity-and-business/>, .
- [30] Richard Nicholson et al. BRAIN-IoT: Model-based framework for dependable sensing and actuation in intelligent decentralized IoT systems. In *IEEE*, October 2019.
- [31] Richard Nicholson et al. Dynamic fog computing platform for event-driven deployment and orchestration of distributed internet of things applications. In *IEEE*, July 2019.

About the Author

Dr. Richard Nicholson is CEO and founder of Tathata Systems Ltd and Chief AI Transformation Officer at Novus-i2, a strategic transformation company based in the United Kingdom. He was the founder and CEO of Paremus Ltd., the company behind Paremus Service Fabric—the continuous OSGi Requirements & Capabilities runtime resolver discussed in Section 6.3 as direct conceptual predecessor to Mycelium’s capability model. First-hand experience designing and deploying Service Fabric over a decade of production use informed both the architectural critique presented in this paper and the design decisions made to move beyond it.

He served as President, Board Director, and Treasurer of the OSGi Alliance for approximately six years [28, 29]. He was awarded the OSGi Laureate (2019) for outstanding contributions to the Alliance and the OSGi Leadership Award (2019) for service as a board officer and advocate for OSGi modularity strategy. He was co-architectural lead for the EU Horizon 2020 Brain-IoT project, a federated platform for emergent coordination across heterogeneous IoT and edge devices [30, 31].